

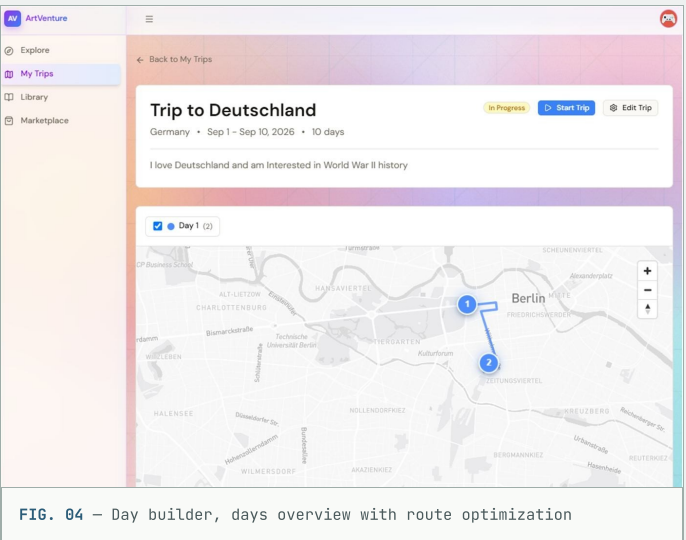
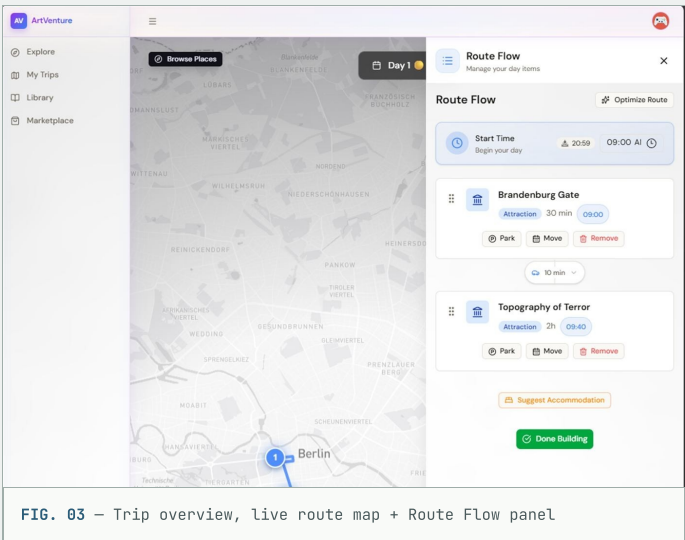
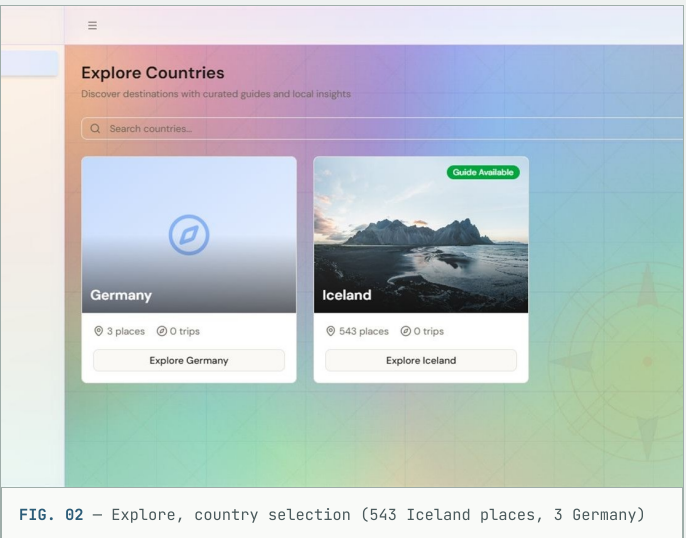
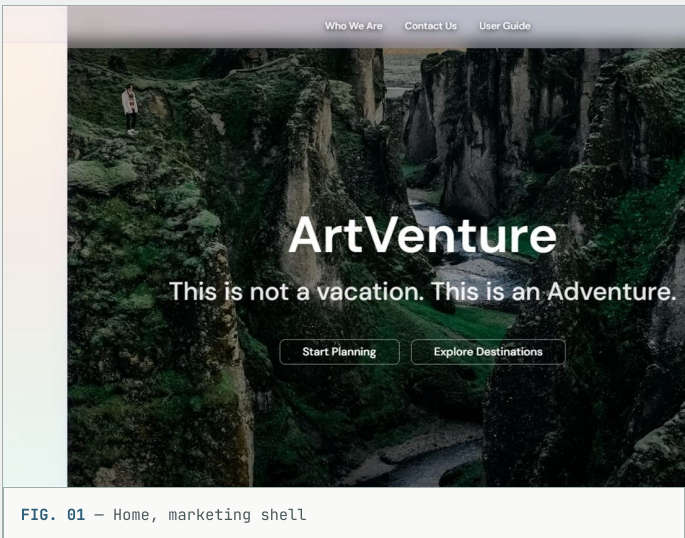
ArtVenture

A trip-planning platform that answers the one question travel apps quietly guess at: **will this day actually work** — given real drive times, real opening hours, and Iceland's extreme daylight swings — computed deterministically, not guessed by an LLM.

74,000	307	48	15	6	152
LINES OF CODE	SOURCE FILES	API ROUTES	DATA MODELS	EXTERNAL APIS	TEST FILES

The product — what it actually looks like

4 VIEWS · CURRENT BUILD



The verdict is the product, not the trip builder. Every planner lets you drag places onto a calendar — none of the mainstream ones tell you, with confidence, whether the day you just built is actually achievable.

Wanderlog, Roadtrippers, and the current wave of AI itinerary generators leave that question unanswered. Press coverage has documented AI planners scheduling arrivals after closing time and activities ending after a flight's departure. ArtVenture's feasibility engine is deterministic: it computes a timeline, it doesn't guess one. Given a day's start time and an ordered list of stops, the engine walks the chain — each arrival is the previous departure plus drive time, each departure is arrival plus that stop's visit duration — and checks it against opening hours, timed-entry bookings, and, in a country where a summer day can run 24 hours and a December one barely 5, sunset.

FEASIBILITY ENGINE — SAMPLE OUTPUT, DAY 1		
09:00	Depart accommodation	day start
09:45	Arrive Seljalandsfoss	45 min drive
10:45	Depart Seljalandsfoss	60 min visit
11:15	Arrive Skógafoss	30 min drive
12:30	Depart Skógafoss	75 min visit
13:20	Arrive ice cave tour (booked 13:00)	RED — 20 MIN LATE

The engine returns **GREEN** / **YELLOW** / **RED** with the specific reason, not just a warning — and it never auto-fixes. The traveler keeps the judgment; the system gives the verdict.

§02

Architecture

FOUR LAYERS, ONE DETERMINISTIC CORE

A conventional full-stack shape carries an unconventional core: a route optimizer and a rules-based verdict engine sitting where most trip apps would just render a map.

FRONTEND	Next.js 16	App Router	React 19	TypeScript	Tailwind CSS 4	Mapbox GL	TanStack Query
BACKEND	48 Next.js API routes, including dedicated feasibility, ETA-chaining, and route-optimizer services, with auth handled by NextAuth 5						
DATA	PostgreSQL (Supabase)	Prisma 7	— 15 models spanning trips, blocks, discovery staging, and quota governance				
COMMERCE	Stripe	checkout, purchases, and promo codes powering a marketplace layer on top of the core planner					

§03

The data pipeline

SIX NOISY SOURCES → ONE TRUSTWORTHY LIBRARY

Real places need real data, and no single free API is complete, accurate, or well-photographed on its own. The discovery pipeline (bLock-maker /, ~7,900 lines) treats that as the actual problem: fetch broadly, reconcile aggressively, spend AI enrichment only on what survives.

Sources — OpenStreetMap / Overpass · Google Places · Foursquare · TripAdvisor · Wikimedia Commons · Gemini (enrichment)

01	02	03	04	05	06
Discover	Deduplicate	Cap	Enrich	Distribute	Review
Query all sources in parallel per region and category, respecting each API's own rate limits.	Cluster candidates by geographic distance and fuzzy name similarity across tiered thresholds, then merge by category-aware source priority.	Rank and truncate each category to a small multiple of its target ceiling before spending AI enrichment on it.	Gemini classifies category, duration, and feasibility per candidate; a quota guardian hard-stops before paid usage without approval.	A grid-based pass trades a little raw quality for spread, so no single cluster crowds out coverage elsewhere.	Everything lands in staging — an admin queue auto-approves high-confidence places and routes the rest to a human.

§04

Engineering craft

PROBLEMS WORTH SOLVING CAREFULLY

DATA QUALITY	COST SAFETY
Confidence-tiered fuzzy matching	A quota guardian for five paid APIs
Duplicate detection runs on a sliding scale — distance and name-similarity	Every external call is checked against a per-service monthly or daily budget with

thresholds tighten together, so a probable match 25 m apart is held to a stricter bar than an exact one 100 m apart. Ambiguous clusters are flagged for review instead of guessed.

GRACEFUL DEGRADATION

Partial success beats total failure

A single exhausted quota used to abort an entire multi-hour discovery run and discard everything already found. Each resolution step now catches that specific failure and continues without the exhausted source instead of losing the run.

real cost estimates, and paid usage past the free tier requires an explicit approval step — the system fails safe, not expensively.

PRODUCT JUDGMENT

Deterministic beats generative, here

The feasibility engine is intentionally not an LLM call. A trip's verdict has to be reproducible and explainable — "20 minutes late for a 13:00 booking" — not a plausible-sounding paragraph that occasionally hallucinates a schedule.

§05 Full stack inventory

FRONTEND

- Next.js 16
- React 19
- TypeScript
- Tailwind CSS 4
- Mapbox GL
- TanStack Query
- Radix UI
- Framer Motion

BACKEND & DATA

- Next.js API Routes
- Prisma 7
- PostgreSQL (Supabase)
- NextAuth 5
- Stripe
- Zod

DATA PIPELINE

- Google Places API
- Foursquare
- TripAdvisor
- OpenStreetMap / Overpass
- Wikimedia Commons
- Gemini API
- Vitest

STATUS

Built iteratively, March–July 2026. The feasibility engine, discovery pipeline, and marketplace layer all work end to end — but this is a working build, not a finished, shipped product.

github.com/roeigam
linkedin.com/in/roei-gamliel
roigamliel@hotmail.com